

# Perfect Network Resilience in Polynomial Time

Matthias Bentert   Stefan Schmid

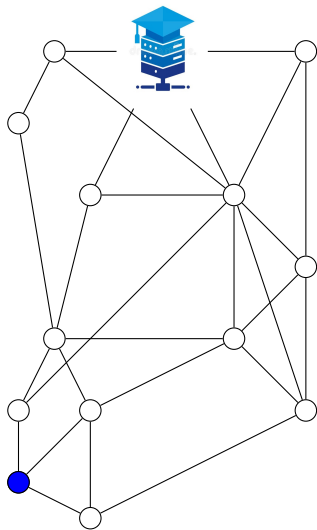
Technische Universität Berlin

Salt Lake City, June 23 2026

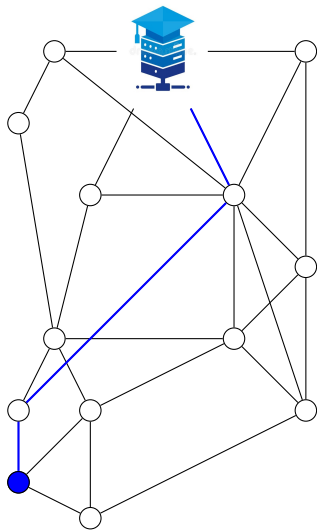
STOC

I just want to push changes...

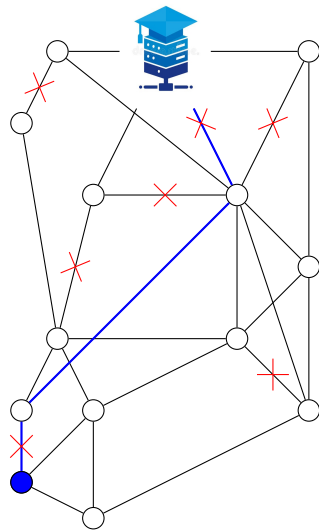
I just want to push changes...



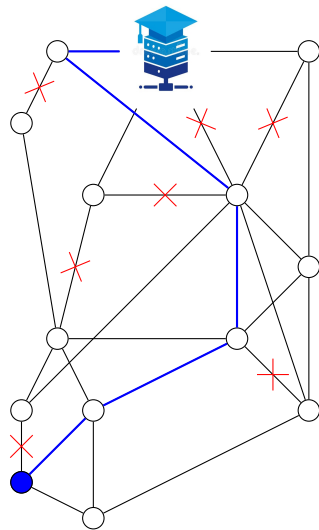
I just want to push changes...



I just want to push changes...

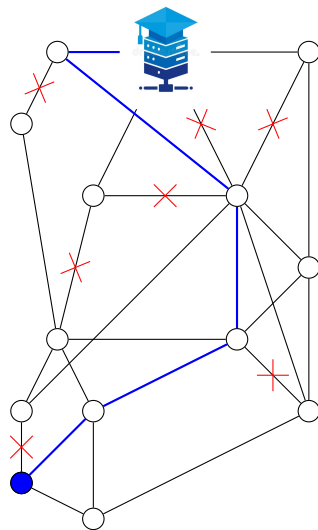


I just want to push changes...



## I just want to push changes...

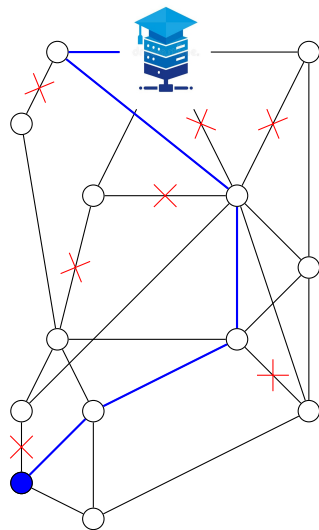
each vertex (router) forwards the packet  
based on limited information:



## I just want to push changes...

each vertex (router) forwards the packet  
based on limited information:

- global information

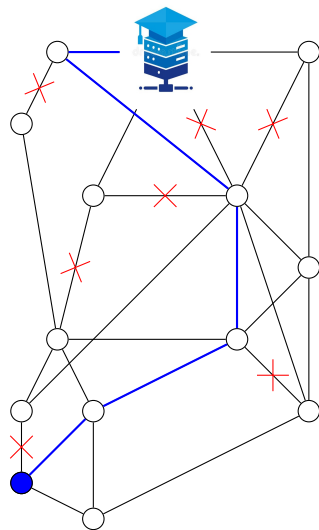




## I just want to push changes...

each vertex (router) forwards the packet  
based on limited information:

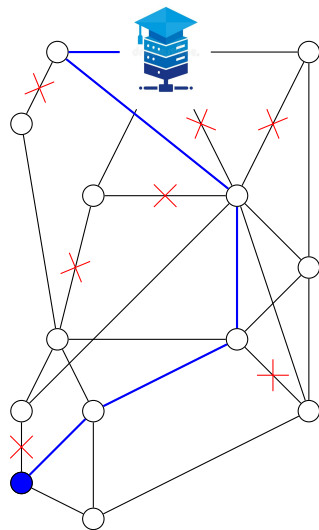
- global information
  - graph (before failures)



## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

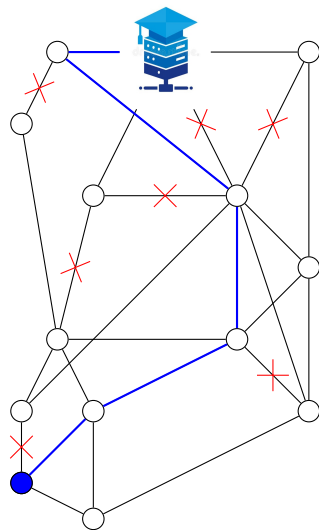
- global information
  - graph (before failures)
  - target  $t$



## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

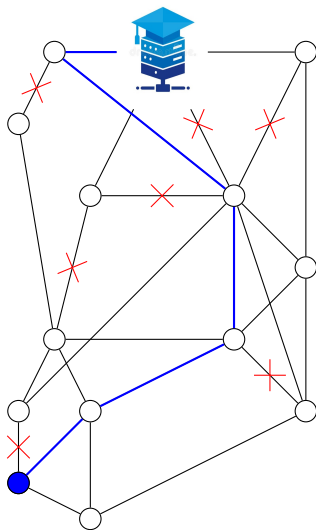
- global information
  - graph (before failures)
  - target  $t$
- local information



## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

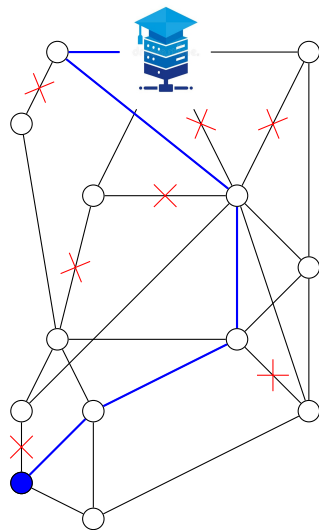
- global information
  - graph (before failures)
  - target  $t$
- local information
  - incident failing connections



## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

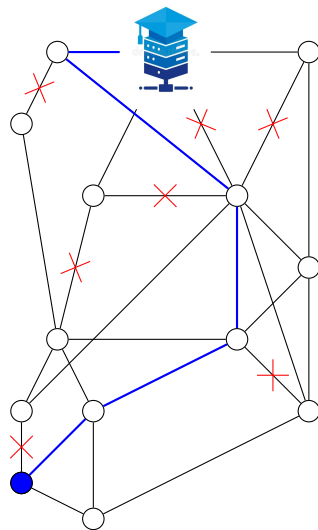
- global information
  - graph (before failures)
  - target  $t$
- local information
  - incident failing connections
  - where did the packet come from (in-port)



## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

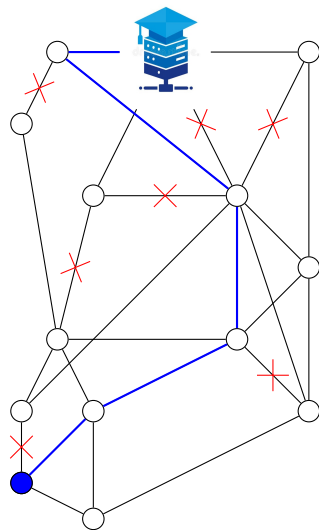
- global information
  - graph (before failures)
  - target  $t$
- local information
  - incident failing connections
  - where did the packet come from (in-port)
- failure model



## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

- global information
  - graph (before failures)
  - target  $t$
- local information
  - incident failing connections
  - where did the packet come from (in-port)
- failure model
  - after setup, edges fail permanently



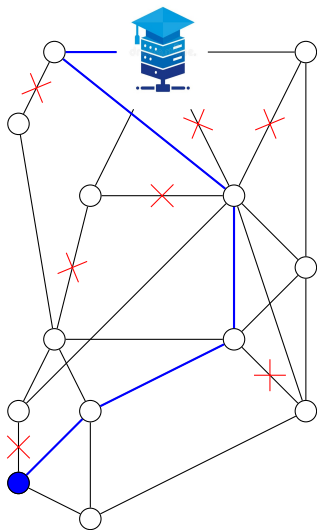
## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

- global information
  - graph (before failures)
  - target  $t$
- local information
  - incident failing connections
  - where did the packet come from (in-port)
- failure model
  - after setup, edges fail permanently

perfect resilience:

resilient against any failures  
(as long as a path to  $t$  remains)





## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

- global information
  - graph (before failures)
  - target  $t$
- local information
  - incident failing connections
  - where did the packet come from (in-port)
- failure model
  - after setup, edges fail permanently

perfect resilience:

resilient against any failures  
(as long as a path to  $t$  remains)

### Perfect Network Resilience Decision

**Input:** A graph  $G$  and a vertex  $t$ .

**Question:** Do perfectly resilient forwarding rules exist?

## I just want to push changes...

each vertex (router) forwards the packet based on limited information:

- global information
  - graph (before failures)
  - target  $t$
- local information
  - incident failing connections
  - where did the packet come from (in-port)
- failure model
  - after setup, edges fail permanently

perfect resilience:

resilient against any failures  
(as long as a path to  $t$  remains)

### Perfect Network Resilience Decision

**Input:** A graph  $G$  and a vertex  $t$ .

**Question:** Do perfectly resilient forwarding rules exist?

### Perfect Network Resilience Synthesis

**Input:** A graph  $G$  and a vertex  $t$ .

**Task:** Design perfectly resilient forwarding rules for each vertex or determine that none exist.

What was (not) known?

## What was (not) known?

Known:

## What was (not) known?

Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pignolet, Schmid, Trédan. APOCS '21]

## What was (not) known?

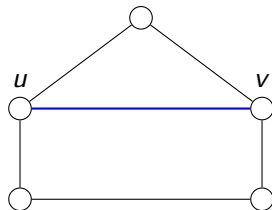
Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pignolet, Schmid, Trédan. APOCS '21]  
(deleting/contracting edges does not break perfect resilience)

## What was (not) known?

Known:

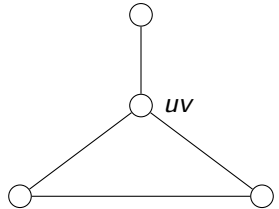
- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pignolet, Schmid, Trédan. APOCS '21]  
(deleting/contracting edges does not break perfect resilience)



## What was (not) known?

Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pignolet, Schmid, Trédan. APOCS '21]  
(deleting/contracting edges does not break perfect resilience)



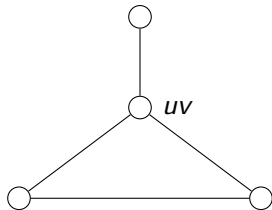


## What was (not) known?

Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pignolet, Schmid, Trédan. APOCS '21]

~→  $n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]



## What was (not) known?

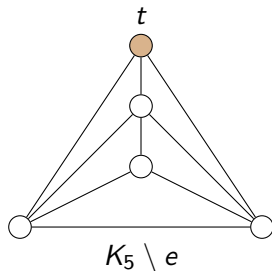
Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pignolet, Schmid, Trédan. APOCS '21]  
     $\rightsquigarrow n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]
- $K_5 \setminus e$  and  $K_{3,3} \setminus e$ : not perfectly resilient  
    [Foerster, Hirvonen, Pignolet, Schmid, Trédan, DSN '22]

## What was (not) known?

Known:

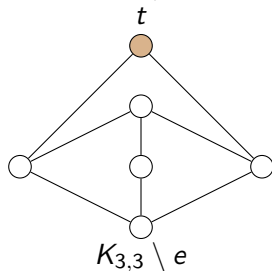
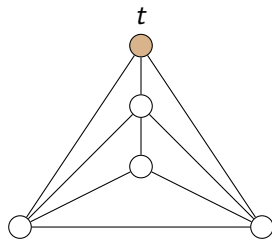
- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]  
     $\rightsquigarrow n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]
- $K_5 \setminus e$  and  $K_{3,3} \setminus e$ : not perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan, DSN '22]



# What was (not) known?

Known:

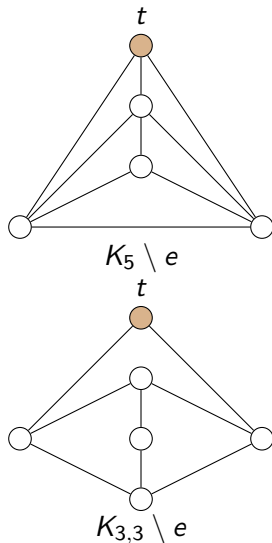
- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]
- $\rightsquigarrow n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]
- $K_5 \setminus e$  and  $K_{3,3} \setminus e$ : not perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan, DSN '22]



# What was (not) known?

Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]  
     $\rightsquigarrow n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]
- $K_5 \setminus e$  and  $K_{3,3} \setminus e$ : not perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan, DSN '22]
- $G - t$  outerplanar: perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]

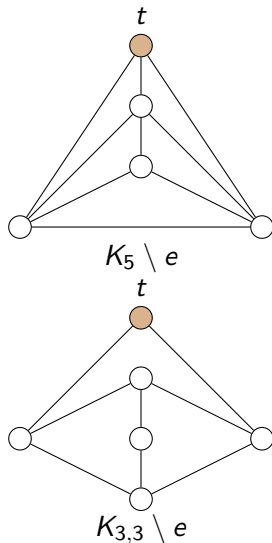


# What was (not) known?

Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]  
     $\rightsquigarrow n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]
- $K_5 \setminus e$  and  $K_{3,3} \setminus e$ : not perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan, DSN '22]
- $G - t$  outerplanar: perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]

Big open problems:



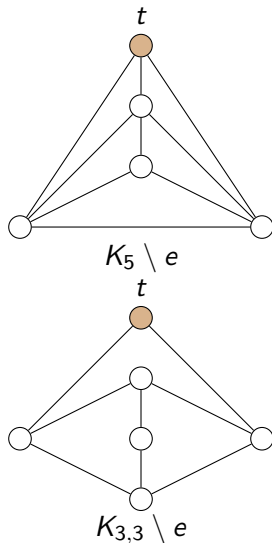
# What was (not) known?

Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]  
     $\rightsquigarrow n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]
- $K_5 \setminus e$  and  $K_{3,3} \setminus e$ : not perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan, DSN '22]
- $G - t$  outerplanar: perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]

Big open problems:

- Which rooted graphs are perfectly resilient?  
    [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]  
    [Feigenbaum, Godfrey, Panda, Schapira, Shenker, Singla. PODC '12]



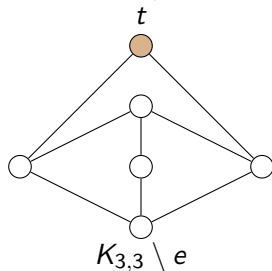
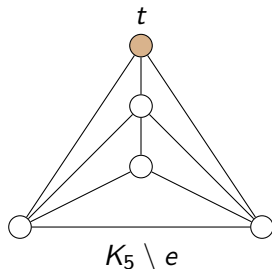
# What was (not) known?

Known:

- **Perfect Resilience Decision** is closed under rooted minors [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]  
→  $n^{1+o(1)}$ -time algorithm [Korhonen, Pilipczuk, Stamoulis. FOCS '24]
- $K_5 \setminus e$  and  $K_{3,3} \setminus e$ : not perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan, DSN '22]
- $G - t$  outerplanar: perfectly resilient [Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]

Big open problems:

- Which rooted graphs are perfectly resilient?  
[Feigenbaum, Godfrey, Panda, Schapira, Shenker, Singla. PODC '12]  
[Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]
- Compact encoding of solutions powerful enough?  
[Chiesa, Nikolaevskiy, Mitrovic, Gurtov, Madry, Schapira, Shenker. ToN '17]  
[Foerster, Hirvonen, Pigolet, Schmid, Trédan. APOCS '21]





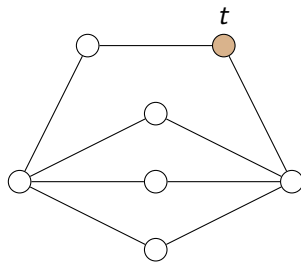
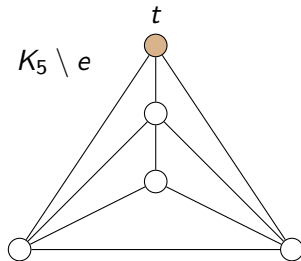
# Our Contribution

## Our Contribution

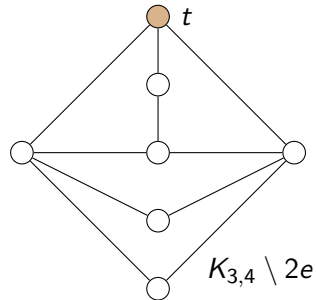
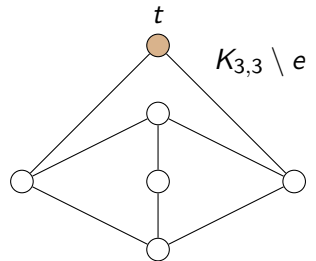
- Complete characterization of forbidden rooted minors

## Our Contribution

- Complete characterization of forbidden rooted minors



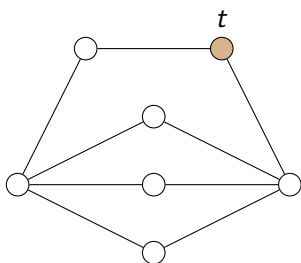
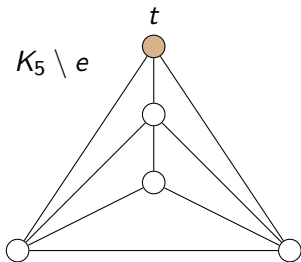
subdivided  $K_{2,4}$



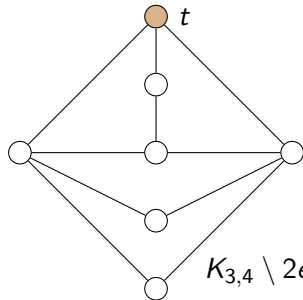
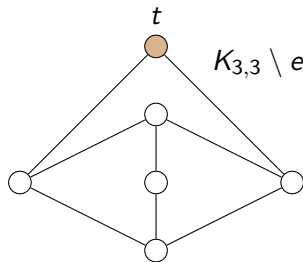
$K_{3,4} \setminus 2e$

## Our Contribution

- Complete characterization of forbidden rooted minors
- $O(n)$ -time algorithm for **Perfect Resilience Decision**



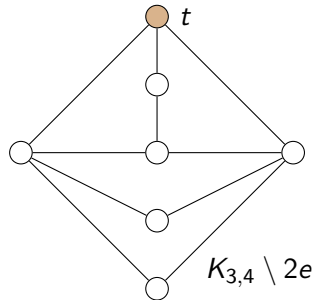
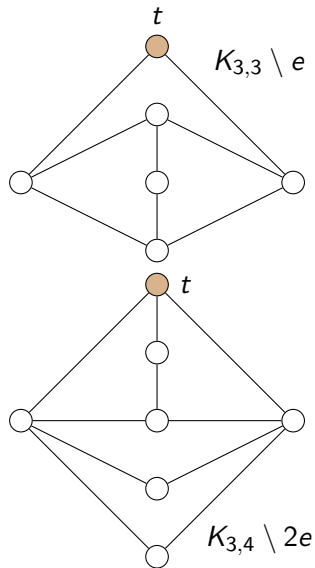
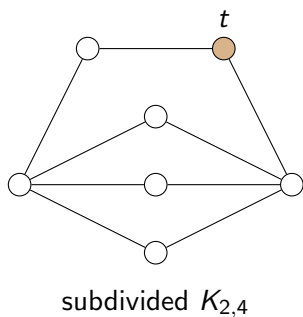
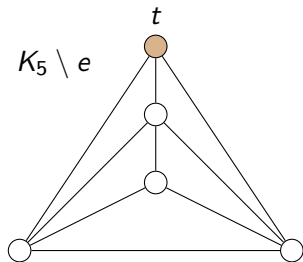
subdivided  $K_{2,4}$



$K_{3,4} \setminus 2e$

## Our Contribution

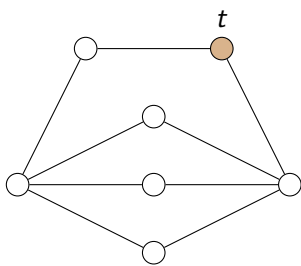
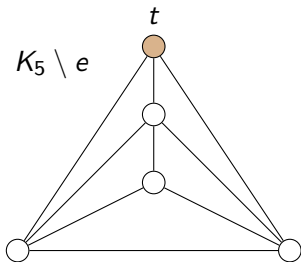
- Complete characterization of forbidden rooted minors
- $O(n)$ -time algorithm for **Perfect Resilience Decision**
- Skipping priority lists are sufficient



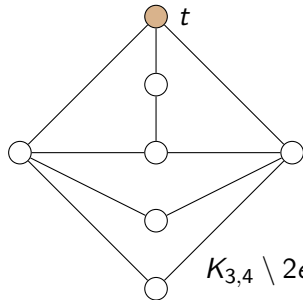
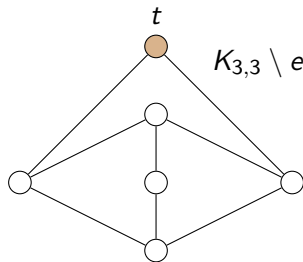
## Our Contribution

- Complete characterization of forbidden rooted minors
- $O(n)$ -time algorithm for **Perfect Resilience Decision**
- Skipping priority lists are sufficient

For each vertex and in-port, store a list of all incident edges and always take first not failing.



subdivided  $K_{2,4}$



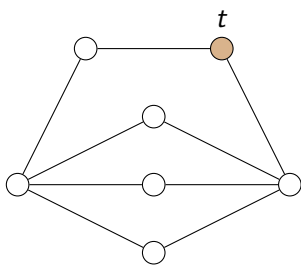
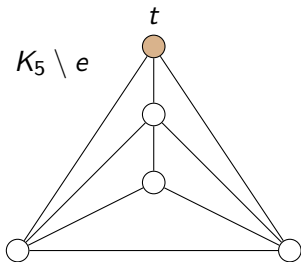
$K_{3,4} \setminus 2e$

## Our Contribution

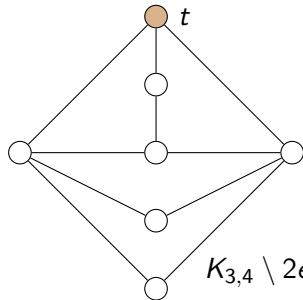
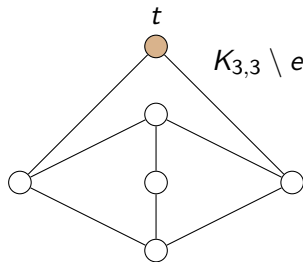
- Complete characterization of forbidden rooted minors
- $O(n)$ -time algorithm for **Perfect Resilience Decision**
- Skipping priority lists are sufficient

For each vertex and in-port, store a list of all incident edges and always take first not failing.

$(\{v, t\}, \{v, u\}, \{v, w\}, \{v, x\})$



subdivided  $K_{2,4}$



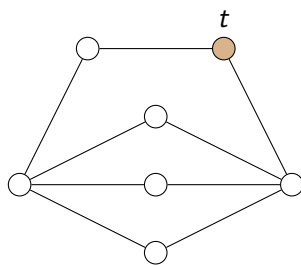
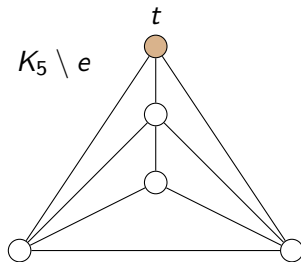
$K_{3,4} \setminus 2e$

## Our Contribution

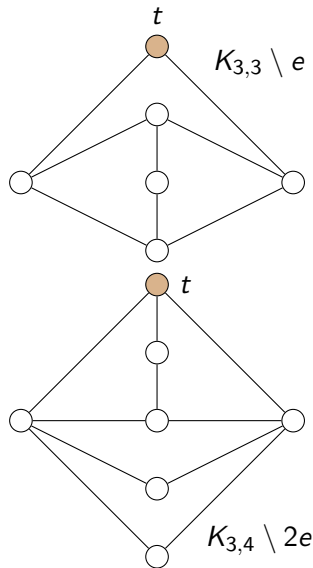
- Complete characterization of forbidden rooted minors
- $O(n)$ -time algorithm for **Perfect Resilience Decision**
- Skipping priority lists are sufficient

For each vertex and in-port, store a list of all incident edges and always take first not failing.

$(\{\cancel{v, t}\}, \{v, u\}, \{\cancel{v, w}\}, \{v, x\})$



subdivided  $K_{2,4}$



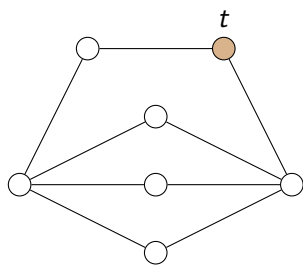
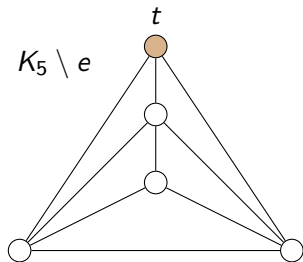


## Our Contribution

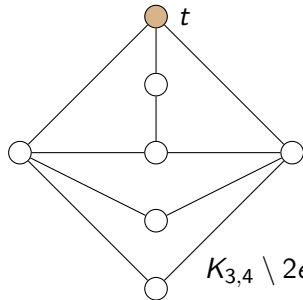
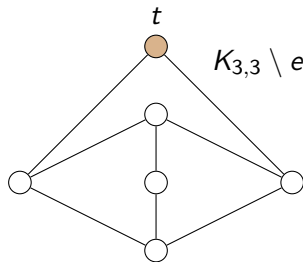
- Complete characterization of forbidden rooted minors
- $O(n)$ -time algorithm for **Perfect Resilience Decision**
- Skipping priority lists are sufficient
- $O(nm)$ -time algorithm for **Perfect Resilience Synthesis**

For each vertex and in-port, store a list of all incident edges and always take first not failing.

$(\{\cancel{v, t}\}, \{v, u\}, \{\cancel{v, w}\}, \{v, x\})$



subdivided  $K_{2,4}$



# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient

# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)

# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Show that each graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- ~~• Show that each graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes~~

# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- ~~• Show that each graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes~~

# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

$\rightsquigarrow$  also yields  $O(nm)$ -time algorithm for **Perfect Resilience Synthesis**



# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

$\rightsquigarrow$  also yields  $O(nm)$ -time algorithm for **Perfect Resilience Synthesis**

$\rightsquigarrow$  also shows that skipping priority lists are sufficient

# High-Level Overview I

## Structural Analysis

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

$\rightsquigarrow$  also yields  $O(nm)$ -time algorithm for **Perfect Resilience Synthesis**

$\rightsquigarrow$  also shows that skipping priority lists are sufficient

# High-Level Overview II

## Linear-Time Algorithm for Decision

- Check whether connected component  $C$  containing  $t$  is planar
  - If not, show that instance is not perfectly resilient

# High-Level Overview II

## Linear-Time Algorithm for Decision

- Check whether connected component  $C$  containing  $t$  is planar
  - If not, show that instance is not perfectly resilient
- Check whether  $C$  has branchwidth at most 9

# High-Level Overview II

## Linear-Time Algorithm for Decision

- Check whether connected component  $C$  containing  $t$  is planar
  - If not, show that instance is not perfectly resilient
- Check whether  $C$  has branchwidth at most 9
  - If so, use  $f(bw) \cdot n$ -time algorithm for finding rooted minors [Courcelle. Inf. Comput. '90]  
[Adler, Dorn, Fomin, Sau, Thilikos. TCS '11]

# High-Level Overview II

## Linear-Time Algorithm for Decision

- Check whether connected component  $C$  containing  $t$  is planar
  - If not, show that instance is not perfectly resilient
- Check whether  $C$  has branchwidth at most 9
  - If so, use  $f(bw) \cdot n$ -time algorithm for finding rooted minors [Courcelle. Inf. Comput. '90]  
[Adler, Dorn, Fomin, Sau, Thilikos. TCS '11]
  - If not, show that instance is not perfectly resilient

# High-Level Overview II

## Linear-Time Algorithm for Decision

- Check whether connected component  $C$  containing  $t$  is planar
  - If not, show that instance is not perfectly resilient
- Check whether  $C$  has branchwidth at most 9
  - If so, use  $f(bw) \cdot n$ -time algorithm for finding rooted minors [Courcelle. Inf. Comput. '90]  
[Adler, Dorn, Fomin, Sau, Thilikos. TCS '11]
  - If not, show that instance is not perfectly resilient

# Structural Analysis



## Overview

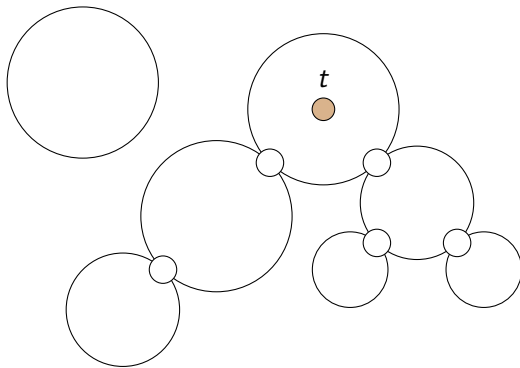
- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

## Overview

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

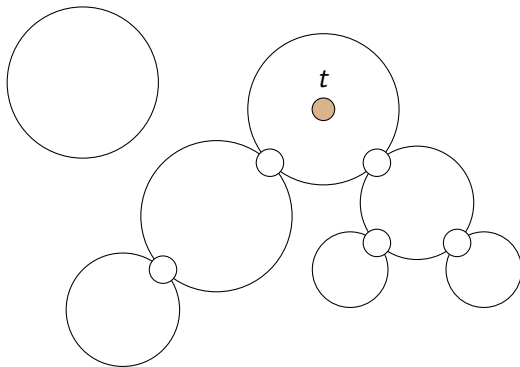
# Preprocessing

## Preprocessing



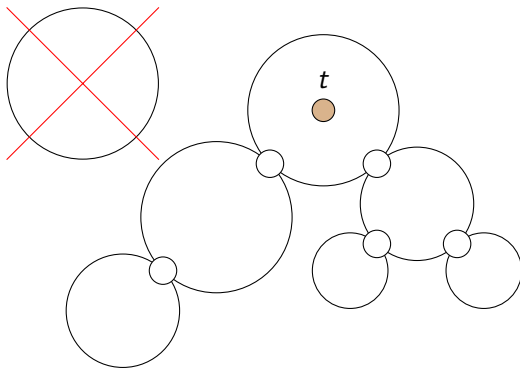
## Preprocessing

- connected



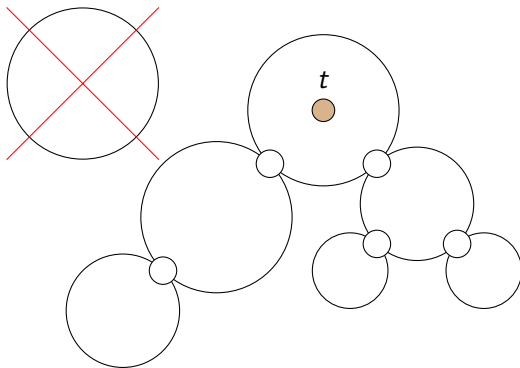
## Preprocessing

- connected



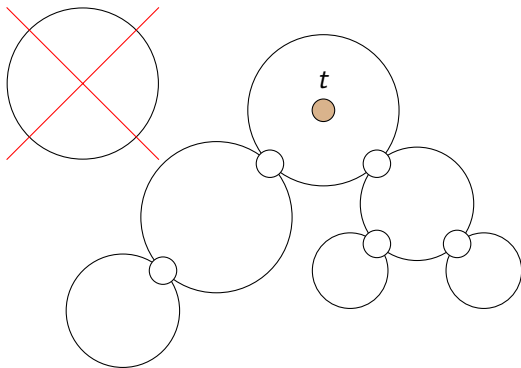
## Preprocessing

- connected
- planar



## Preprocessing

- connected
- planar



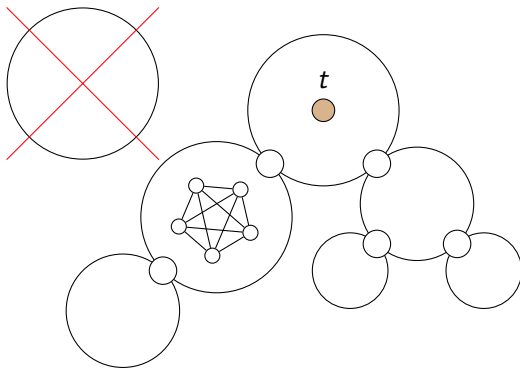
Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*



## Preprocessing

- connected
- planar



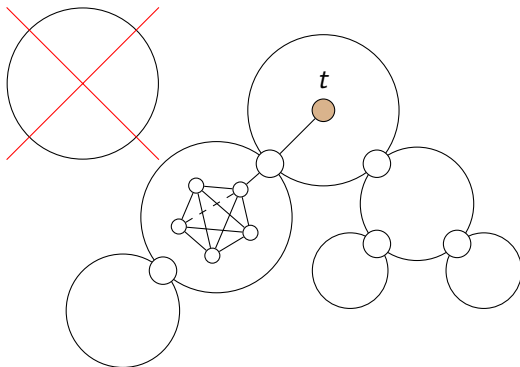
Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*



## Preprocessing

- connected
- planar

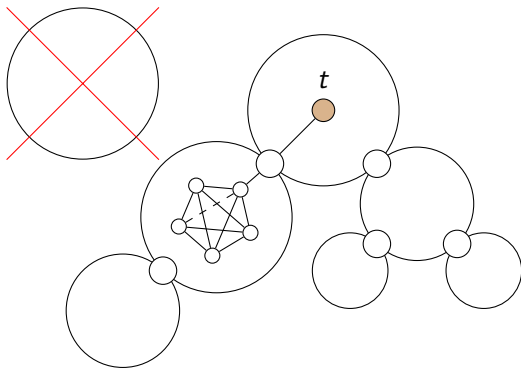


Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*

## Preprocessing

- connected
- planar
- biconnected

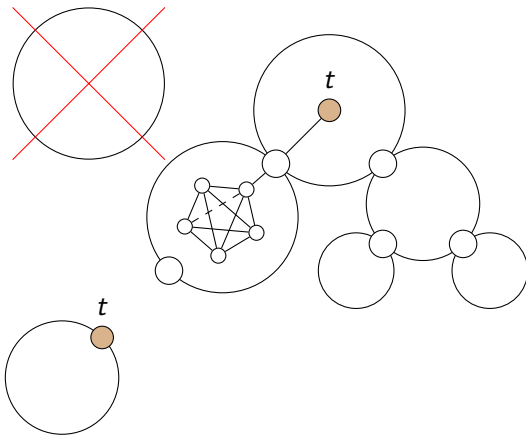


Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*

## Preprocessing

- connected
- planar
- biconnected

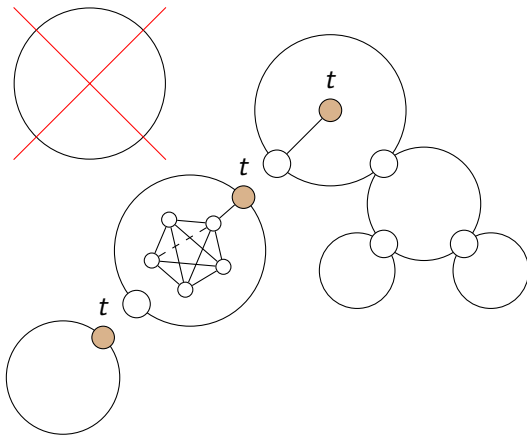


Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*

## Preprocessing

- connected
- planar
- biconnected

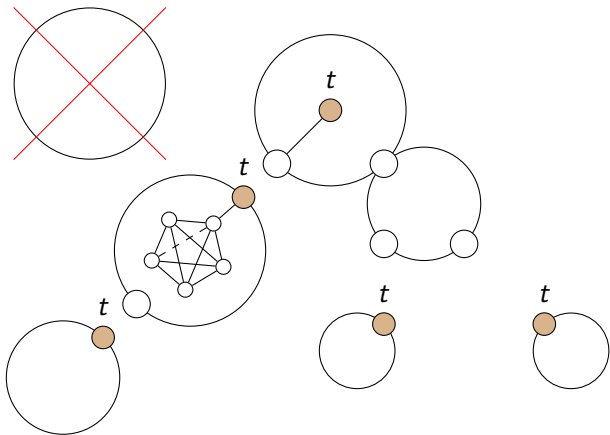


Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*

## Preprocessing

- connected
- planar
- biconnected

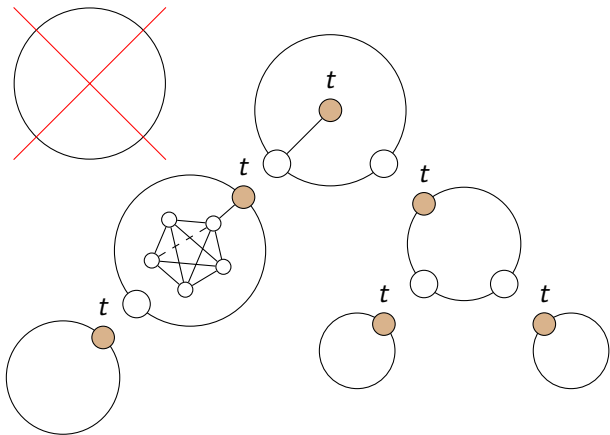


Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*

## Preprocessing

- connected
- planar
- biconnected



Theorem (Wagner. Math. Ann. '37)

*A graph  $G$  is planar if and only if it does not contain  $K_5$  or  $K_{3,3}$  as a minor.*



## Overview

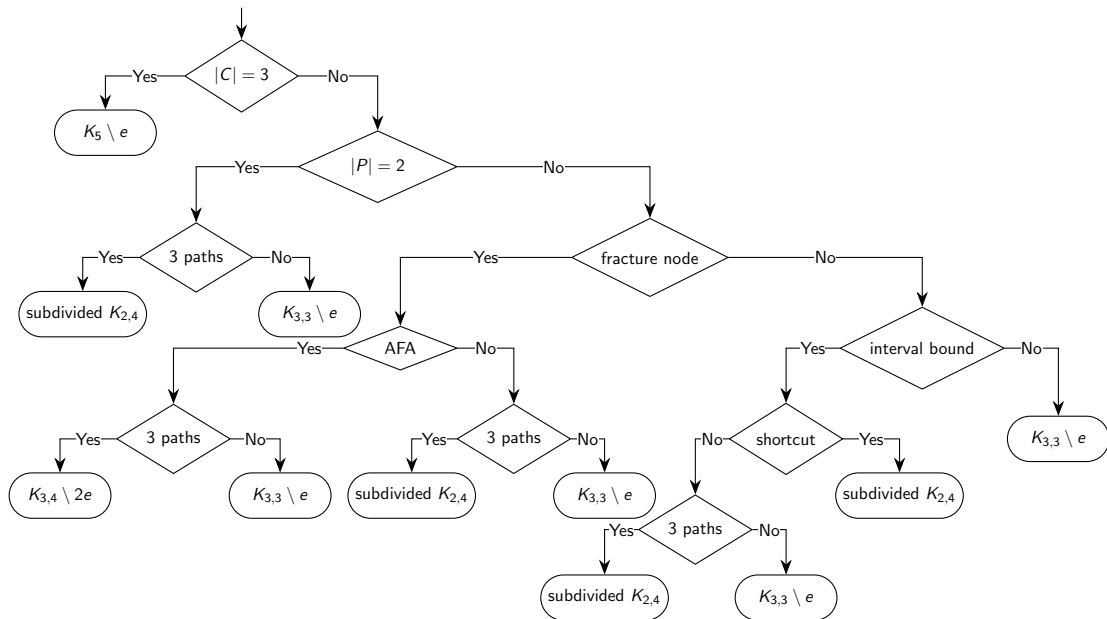
- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

## Overview

- Show that  $K_{3,4} \setminus 2e$  and subdivided  $K_{2,4}$  are not perfectly resilient
- Show that two new classes of rooted graphs are perfectly resilient  
(with skipping priority lists)
- Ensure additional properties via preprocessing
- Show that each preprocessed graph that does not contain any of the four rooted obstructions belongs to one of the three graph classes

## Glimpse Into the Proof

## Glimpse Into the Proof



## Linear-Time Algorithm for Decision

# Overview

- Check whether connected component  $C$  containing  $t$  is planar
  - If not, show that instance is not perfectly resilient
- Check whether  $C$  has branchwidth at most 9
  - If so, use  $f(bw) \cdot n$ -time algorithm for finding rooted minors [Courcelle. Inf. Comput. '90]  
[Adler, Dorn, Fomin, Sau, Thilikos. TCS '11]
  - If not, show that instance is not perfectly resilient

# Overview

- Check whether connected component  $C$  containing  $t$  is planar
  - If not, show that instance is not perfectly resilient
- Check whether  $C$  has branchwidth at most 9
  - If so, use  $f(bw) \cdot n$ -time algorithm for finding rooted minors [Courcelle. Inf. Comput. '90]  
[Adler, Dorn, Fomin, Sau, Thilikos. TCS '11]
  - If not, show that instance is not perfectly resilient

# Linear Time Algorithm



# Linear Time Algorithm

Theorem (Gu and Tamaki. Algorithmica '12)

*Let  $G$  be a planar graph and let  $g$  be the largest integer such that  $G$  contains a  $g \times g$  grid as a minor. Then, the branchwidth of  $G$  is at most  $3g$ .*

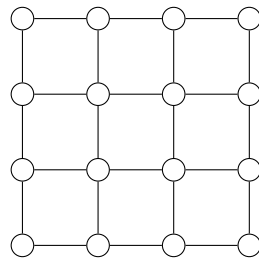
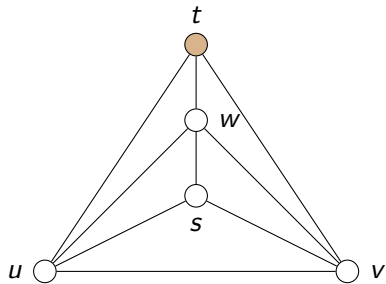
# Linear Time Algorithm

Theorem (Gu and Tamaki. Algorithmica '12)

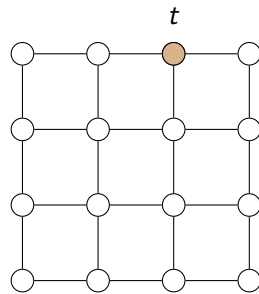
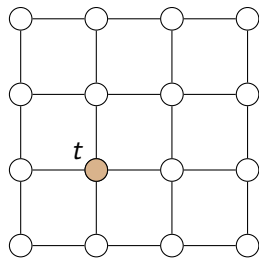
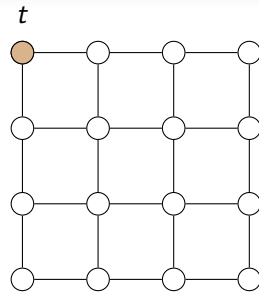
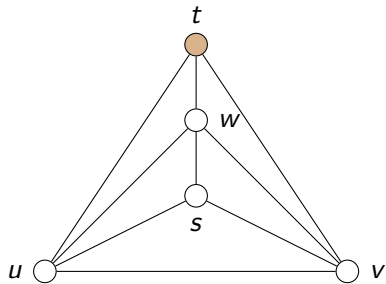
*Let  $G$  be a planar graph and let  $g$  be the largest integer such that  $G$  contains a  $g \times g$  grid as a minor. Then, the branchwidth of  $G$  is at most  $3g$ .*

$\rightsquigarrow$  Connected component of  $t$  has branchwidth at most 9 or contains 4x4-grid as minor

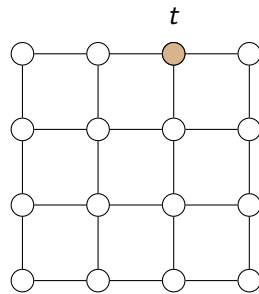
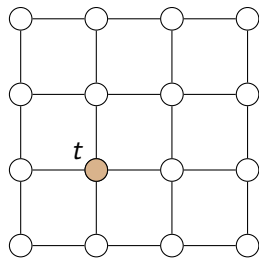
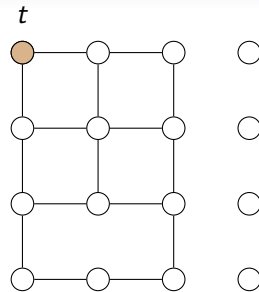
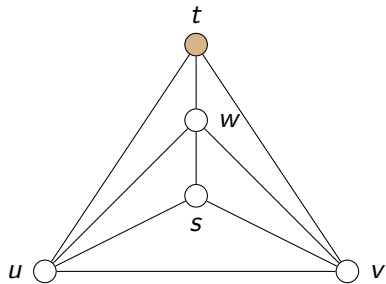
## Grids



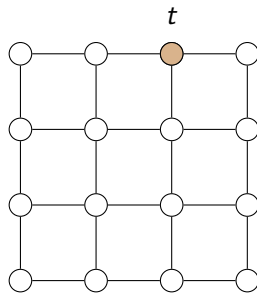
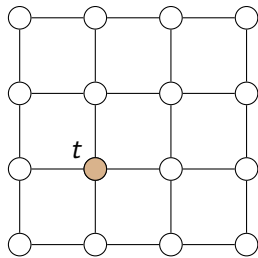
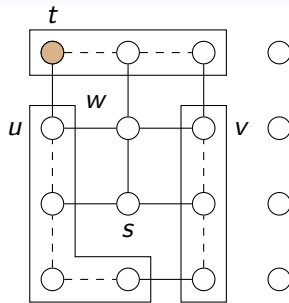
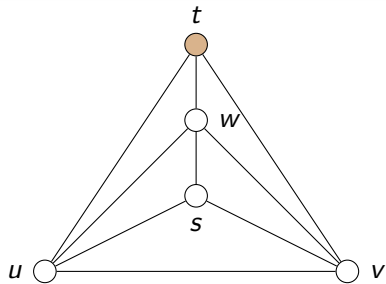
## Grids



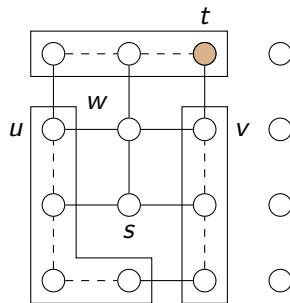
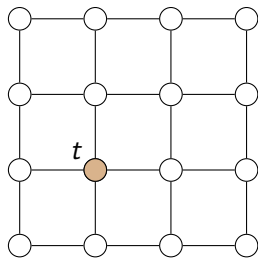
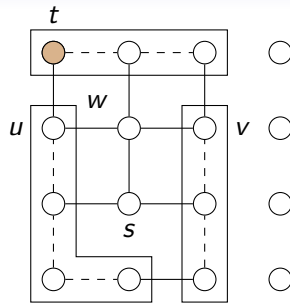
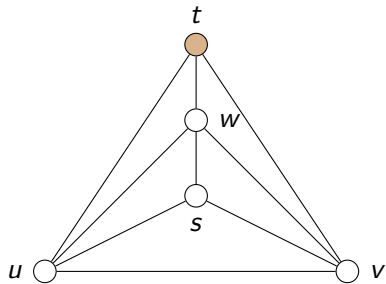
## Grids



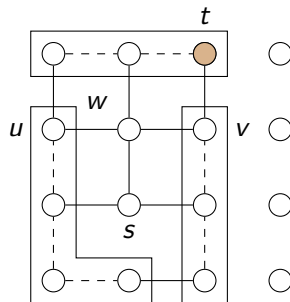
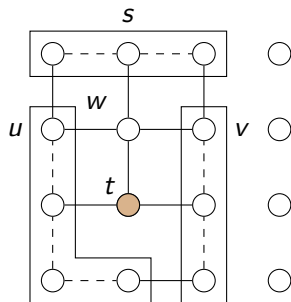
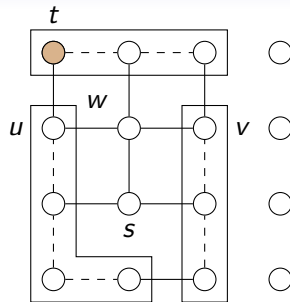
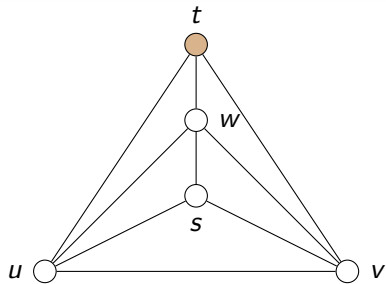
## Grids



## Grids



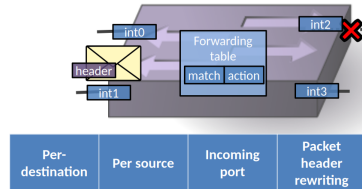
## Grids





## Future Work

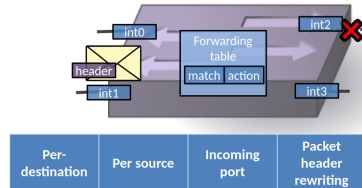
# Future Work



# Future Work

## 1. Additional Information

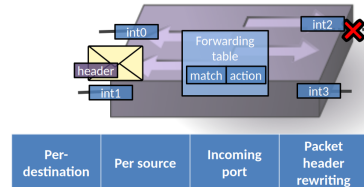
- Source vertex



# Future Work

## 1. Additional Information

- Source vertex
- Modifiable header bits



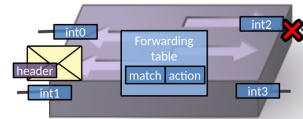
# Future Work

## 1. Additional Information

- Source vertex
- Modifiable header bits

## 2. Different Failure Models

- (Semi-)dynamic edge failures



| Per-destination | Per source | Incoming port | Packet header rewriting |
|-----------------|------------|---------------|-------------------------|
|-----------------|------------|---------------|-------------------------|

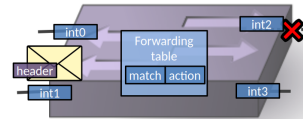
# Future Work

## 1. Additional Information

- Source vertex
- Modifiable header bits

## 2. Different Failure Models

- (Semi-)dynamic edge failures
- Vertex failures



| Per-destination | Per source | Incoming port | Packet header rewriting |
|-----------------|------------|---------------|-------------------------|
|-----------------|------------|---------------|-------------------------|

# Future Work

## 1. Additional Information

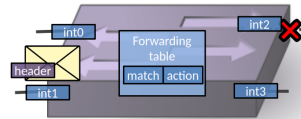
- Source vertex
- Modifiable header bits

## 2. Different Failure Models

- (Semi-)dynamic edge failures
- Vertex failures

## 3. Weaker Guarantee

- Ideal resilience (any  $k$ -connected graph is  $(k - 1)$ -resilient)



| Per-destination | Per source | Incoming port | Packet header rewriting |
|-----------------|------------|---------------|-------------------------|
|-----------------|------------|---------------|-------------------------|

# Future Work

## 1. Additional Information

- Source vertex
- Modifiable header bits

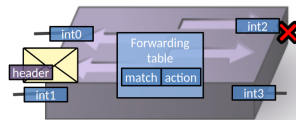
## 2. Different Failure Models

- (Semi-)dynamic edge failures
- Vertex failures

## 3. Weaker Guarantee

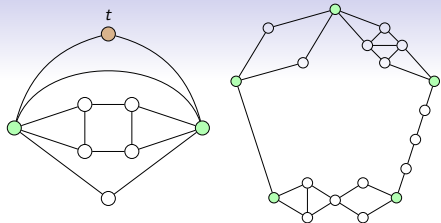
- Ideal resilience (any  $k$ -connected graph is  $(k - 1)$ -resilient)

Thank you.

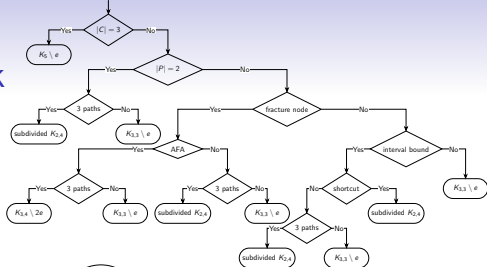


| Per-destination | Per source | Incoming port | Packet header rewriting |
|-----------------|------------|---------------|-------------------------|
|-----------------|------------|---------------|-------------------------|





## Future Work



### 1. Additional Information

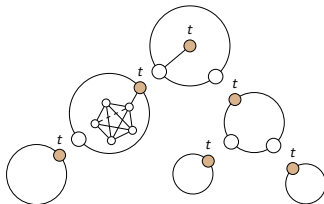
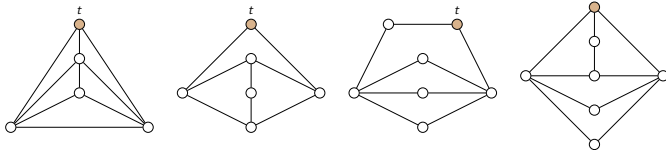
- Source vertex
- Modifiable header bits

### 2. Different Failure Models

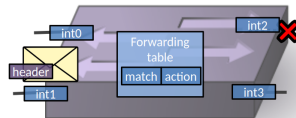
- (Semi-)dynamic edge failures
- Vertex failures

### 3. Weaker Guarantee

- Ideal resilience (any  $k$ -connected graph is  $(k - 1)$ -resilient)



Thank you.



| Per-destination | Per source | Incoming port | Packet header rewriting |
|-----------------|------------|---------------|-------------------------|
|-----------------|------------|---------------|-------------------------|